Architectural Blueprint and Comprehensive Brand Guidelines: Transitioning the Hourglass Application to a Cross-Platform Spatial Dark Glass User Interface

1. Executive Summary and Strategic Alignment

An exhaustive architectural and aesthetic review of the "Hourglass" application reveals a critical juncture in its product lifecycle and technical execution. The application currently functions as a highly specialized work dashboard tailored for crossover contractors and project managers.[1] Its primary mission is to instantly answer the user's implicit question—"Where does my week stand, and where is it heading?"—in under three seconds of cognitive processing time.[1] While the existing structural layout successfully establishes a logical data hierarchy, its reliance on a legacy interpretation of "dark mode"—characterized by flat, heavily opaque geometrical surfaces, static vector graphics, and rudimentary data visualization rendering—fails to meet the sophisticated expectations of contemporary digital product design.[1]

In the modern mobile ecosystem, driven by the capabilities of frameworks such as React Native 0.83 and Expo SDK 55, user interface expectations have shifted aggressively toward spatial computing and physical material paradigms.[1] Premium applications no longer render static, unilluminated pixels on a two-dimensional flat plane. Instead, they simulate physical, translucent materials that interact dynamically with environmental lighting, user physics, and z-axis depth.[1] To elevate the Hourglass application to a "confident, precise, and premium" aesthetic comparable to industry leaders like the Oura Ring application, Linear, and Arc Browser, a complete overhaul of both the Brand Guidelines and the underlying technical architecture is strictly required.[1]

The strategic objective of this comprehensive document is to fulfill two highly interconnected mandates. First, it establishes the finalized **Hourglass Brand Guidelines (Version 2.0)**, redefining the application's visual identity around a "modern, flowy, dark glass, tech" aesthetic.[1] Second, it provides an exhaustive, constraint-aware **Technical Migration Specification**. Because the engineering team has encountered friction transitioning the current codebase to these new aesthetic paradigms, this specification details the exact methodologies required to migrate from bridge-reliant React Native User Interface (UI) components to hardware-accelerated rendering pipelines. This transition dictates the adoption of React Native Skia for graphics, dynamic Skia Shading Language (SKSL) fragment shaders, physics-based

Reanimated 4.2 Worklets, and high-performance charting via Victory Native XL, ensuring absolute feature parity across both iOS and Android devices via the Expo framework.[1]

2. Hourglass Brand Guidelines (Version 2.0)

The updated brand guidelines pivot fundamentally away from static hexadecimal application toward a dynamic, material-first rendering approach. In a spatial dark glass system, the brand identity is communicated not merely through static color codes and typography, but through the simulation of physical behaviors, light refraction, volumetric shadows, and the emulation of physical depth.

2.1 The Deep Space Palette and Semantic Lensing

The foundational philosophy of the Hourglass palette relies on maintaining a deep, hued-black canvas infused with a subtle eggplant undertone.[1] This specific chromatic choice is deliberately engineered to prevent the severe visual ocular fatigue typically associated with pure black (#000000) environments, while simultaneously causing high-energy semantic accent colors to radiate with increased perceived warmth and luminosity.[1] However, in Version 2.0, these foundational colors are no longer applied as flat backgroundColor cascading stylesheet (CSS) properties. Instead, they serve as the mathematical inputs that dictate environmental lighting shaders and the tinting of refractive glass materials.

The following table delineates the foundational palette transformation from flat rendering to spatial material application:

Palette Role	Base Hexadecimal	Material Application in Version 2.0 Architecture	Contextual and Semantic Meaning
Environmental Background	#0D0C14 (Deep Eggplant)	Serves as the anchor base color for the underlying animated SKSL topological mesh gradient.	Represents the infinite spatial void behind the user interface, injecting subtle warmth.[1]
Default Surface Container	#16151F	Replaced by a BackdropFilter utilizing a 16px to 20px blur radius,	Represents the default glass analytical cards resting directly

		overlaid with a subtle white noise texture at 0.03 opacity.	above the environmental mesh.[1]
Elevated Surface / Modal	#1F1E29	Replaced by a multi-layered BackdropFilter with increased light exposure and a 30px blur radius.	Utilized exclusively for z-axis elevation, such as modals, bottom sheets, and active PressIn interactive states.[1]
Container Borders	#2F2E41	Replaced by a microscopic, hardware-accelerated Masked LinearGradient spanning 1px to 1.5px.	Defines the chamfered, physical edge of the glass pane catching the ambient environmental light.[1]
Muted Metadata	#484F58	Mathematical desaturation applied to secondary typography strings to recede into the glass.	Utilized for placeholders, disabled states, and non-critical timestamp markers.[1]

Semantic Accent Color Governance

Accent colors within the Hourglass ecosystem are strictly regulated. They must act as the absolute focal points of the interface, ensuring that the surrounding glass UI acts merely as a non-competing frame for the analytics.[1]

- **Financial Gold (#E8C97A / #FFDF89)**: Reserved strictly and exclusively for financial metrics, monetary earnings, and the ultimate "Crushed It" weekly state indicator.[1] The application of this gold metric for generic UI decoration or non-earnings elements is expressly prohibited, as it dilutes the neurological reward associated with financial achievement.
- **Technological Cyan (#00C2FF)**: This electric hue serves as the definitive technological accent, reserved exclusively for Artificial Intelligence (AI) usage metrics, AI trajectory line charts, and AI-driven predictive insights.[1]

- **Deep-Work Violet (#A78BFA)**: Serving as the primary interactive accent, this vibrant purple is utilized for BrainLift hours, deep-work analytical tracking, primary call-to-action (CTA) buttons, and focused navigation indicator states.[1]
- **Status Identifiers**: The week pace states are mapped directly to universal status indicators. success utilizes #10B981 (Green), warning utilizes #F59E0B (Orange), and critical utilizes #F43F5E (Red).[1] A distinct destructive red (#F85149) is isolated specifically for irreversible user actions, such as deleting data or rejecting proposals.[1]

2.2 Dynamic Panel States and Volumetric Subsurface Glows

Version 1.1 of the brand guidelines directed developers to utilize basic radial gradients emanating from the center of analytical cards to signal the weekly status of the user.[1] While logically sound, the visual execution of painting a flat radial gradient behind text destroys the three-dimensional illusion of the tech aesthetic. Version 2.0 modernizes this fundamental concept by entirely replacing flat background gradients with volumetric, colored subsurface glows.

These glows are mathematically emitted from the data points themselves and are programmed to visually refract through the frosted glass surfaces of the containers.[1] The interface communicates the user's weekly trajectory through the following luminous state machine:

- **On Track (Pace ≥ Target)**: The card's underlying Skia canvas emits a smooth, diffused success green (#10B981) glow at approximately 35% opacity.[1] This creates the illusion that the data point is radiating positive energy into the glass.
- **Behind (Pace 50% – 99% of Target)**: The canvas shifts its mathematical output to emit a warning orange (#F59E0B) gradient at 35% opacity.[1]
- **Critical (Pace < 50% of Target)**: The canvas emits an urgent, pulsing critical red (#F43F5E) glow.[1]
- **Crushed It (Target Fully Met)**: The interface transitions to the aforementioned financial gold (#E8C97A) gradient glow.[1]
- **Idle / Weekend**: The panel emits no localized light, relying entirely on the default refraction of the underlying environmental spatial mesh passing beneath the #16151F equivalent frosted glass.[1]

2.3 Typographical Engineering for Spatial Dark Environments

Typography within a dark, glassmorphic interface requires meticulous optical calibration. A critical architectural failure that renders the legacy Hourglass application "stale" is the reliance on standard operating system fonts (such as San Francisco or Roboto) rendered with pure white (#FFFFFF) or near-white text directly on dark, opaque backgrounds.[1] This high-contrast combination triggers a neurological optical phenomenon known as halation, wherein intensely bright typographical forms bleed visually into the surrounding dark environment. This causes rapid user eye strain, reduced numerical legibility, and an overarching perception of unrefined,

clumsy interface design.[1]

To achieve the "confident, precise, premium" tech aesthetic demanded by the guidelines, the application must completely transition to specialized variable and static fonts optimized specifically for complex numerical displays and high-legibility geometric structures.[1]

The Typographical Pairing Matrix

Standard system fonts natively lack the distinct personality required to elevate the interface. The architecture must integrate Google Fonts via the @expo-google-fonts modular packages to ensure production-ready, cross-platform typographical rendering.[1]

The following table dictates the definitive typographical selections for the Hourglass ecosystem:

Typographical Role	Assigned Typeface	Aesthetic Rationale and Technical Function
Primary Headings & Hero Metrics (3xl, xl)	**Space Grotesk**	A geometric, slightly brutalist sans-serif that operates as the definitive typeface for technical, AI-driven, and data-heavy dashboards.[1] It provides immediate visual impact for the 36px hero numbers.
Secondary Labels & Data Tables (md, xs)	**Space Mono**	A highly structured monospaced typeface that acts as a complementary partner to Space Grotesk.[7] It is strictly utilized for complex data tables where vertical column alignment of unpredictable numerical widths is mandatory for rapid cognitive parsing.
Neutral Body Copy (md, xs)	**Inter**	The undisputed gold standard for neutral, high-legibility digital interfaces.[1] Featuring a

		massive x-height, it ensures absolute clarity for long-form reading and metadata at microscopic scaling sizes (12px to 16px).[1]

Strict Optical Calibration Rules

To seamlessly blend the typography into the liquid glass surfaces, engineering teams must execute the following rules:

1. **Mathematical Desaturation**: Typography must never utilize pure white (#FFFFFF). Primary hero numbers must utilize a slightly muted grey (#E0E0E0); secondary metadata and body copy must utilize #A0A0A0 (replacing the legacy #8B949E); and disabled, placeholder, or muted states must utilize #757575 (replacing the legacy #484F58).[1] This desaturation completely eliminates the visual vibration caused by intense contrast.[1]
2. **Optical Weight Reduction**: Light-emitting mobile displays inherently render light text on dark backgrounds as optically thicker than the inverse scenario. Developers must counteract this illusion by strictly utilizing lighter font weights in the codebase (e.g., dropping from a Display 800 weight to a Display 700 weight, or from a 400 Regular weight to a 300 Light weight) to maintain the intended delicate geometry of the typefaces.[1]
3. **Tabular Numerics**: Because the primary mission of the app is to present live financial and time-tracking metrics, all numeric values must enforce the font-variant-numeric: tabular-nums CSS equivalent.[1] This guarantees that all numbers consume identical horizontal space, preventing the interface from violently jittering along the x-axis when a clock ticks from "11.1h" to "11.8h". Furthermore, letter-spacing (tracking) must be tightened to -0.02em at Display sizes to create compact, authoritative metric blocks.[1]

2.4 Spatial Layout, Geometry, and Edge Illumination

A modern flowy interface relies heavily on empty space to allow the underlying animated background meshes to breathe and interact with the glass. The spatial layout system strictly adheres to a Tailwind-inspired 4px base mathematical scale.[1]

- **Internal Card Padding**: All liquid glass containers must utilize generous internal padding, specifically standardizing on p-5 (20px) or p-6 (24px) to ensure data elements are not claustrophobically pressing against the refractive borders.[1]
- **Horizontal Viewport Margins**: Screen layouts must strictly maintain px-4 (16px) horizontal padding, ensuring a distinct physical separation between the edge of the glass cards and the physical bezel of the mobile device hardware.[1]

Corner Smoothing and Border Geometry

Sharp corners are antithetical to physical liquid glass. The application must enforce a strict

border-radius hierarchy:

- Large analytical cards and main structural containers: rounded-2xl (16px).[1]
- Interactive elements, inputs, and primary buttons: rounded-xl (12px).[1]
- Inner nested elements (e.g., small status badges inside a card): rounded-lg (8px).[1]
- *Strict Prohibition*: Any border radius of rounded-md (6px) or smaller is explicitly prohibited across the application, as it instantly destroys the tactile, molded aesthetic of premium hardware materials.[1]

Edge Illumination Engineering

A highly common pitfall in implementing dark mode glassmorphism is failing to define the structural boundaries of the translucent elements. If a blurred glass card is placed directly against a dark mesh background without specific edge illumination, the card bleeds into the environment, rendering the interface muddy and disorganized.[1] Currently, the Hourglass application relies on flat, 1-pixel solid grey borders.[1]

In Version 2.0, this flat border must be entirely eradicated. A piece of physical glass catching environmental light exhibits a luminous, gradient edge and deep inner shading.[1] All primary cards must be framed by a hardware-accelerated gradient border, transitioning from the brand's primary Violet (#A78BFA) or respective status color to complete transparency, simulating a chamfered edge reflecting a localized light source.[1]

---

3. Technical Migration Specification: The Cross-Platform Spatial Architecture

Transforming the stale, flat architecture of the current Hourglass application into the definitively modern, flowy, dark glass environment mandated by Version 2.0 of the Brand Guidelines is not an exercise in superficial CSS styling. Because the user requires this application to work seamlessly across both iOS and Android devices via the Expo framework, the engineering team must execute a foundational architectural overhaul.[1]

The application must abandon standard React Native View components and native UI bridging in favor of direct, GPU-accelerated graphics rendering, physics-based motion choreography, and high-performance Skia charting.[1]

Phase I: Ecosystem Upgrades and Framework Initialization

The migration fundamentally relies on the advanced capabilities introduced in the latest iteration of the React Native ecosystem. The engineering team must execute the following infrastructure upgrades:

1. **Expo SDK 55 and React Native 0.83.1**: The project must be upgraded to Expo SDK 55,

which bundles React Native 0.83.1 and React 19.2.[2] SDK 55 officially drops the legacy architecture, forcing the application to operate on the highly performant New Architecture (Fabric Renderer and JavaScript Interface).[2] This upgrade is non-negotiable, as React Native 0.83 provides the critical Web Performance APIs and Performance Panel tracking necessary to profile the heavy GPU workloads generated by glassmorphism.[3]

2. **Hermes Engine and Bytecode Diffing**: Ensure the Hermes v1 JavaScript engine is explicitly enabled. Furthermore, to counteract the immense bundle size increase caused by importing the Skia C++ rendering engine (approximately 400KB), the team must enable Hermes Bytecode Diffing for Expo Updates. This optimization slashes Over-The-Air (OTA) download sizes by an estimated 75%, maintaining rapid deployment capabilities.[1]
3. **Dependency Purge**: Strip all legacy, bridge-reliant dependencies from the package.json. Specifically, react-native-chart-kit, basic react-native-svg implementations, and pre-v40 versions of victory-native must be uninstalled, as they will cause catastrophic UI thread blocking during complex animations.[1]

Phase II: Constructing the Mathematical Rendering Engine (Flowy Background)

The defining characteristic of the tech aesthetic is the "flowy" background. Currently, the application relies on a solid dark background with a static warm gradient injected behind the main header, completely shattering the illusion of a living technical environment.[1]

To achieve a fluid background with strict cross-platform parity across iOS and Android, developers must not rely on native OS modules like iOS's expo-ios-mesh-gradient, as Android natively lacks an identical counterpart.[10] Instead, the architecture must leverage @shopify/react-native-skia to construct the background using the Skia Shading Language (SKSL) or highly choreographed declarative gradient primitives.[1]

SKSL Fragment Shader Implementation

The most performant approach is to execute complex mathematical functions per-pixel directly on the mobile GPU, bypassing the React Native JavaScript thread entirely.[1]

1. Construct a RuntimeEffect at the root layout of the application.
2. Pass dynamic variables, known as "uniforms," from a Reanimated 4.2 Worklet into the SKSL shader program.[1] These uniforms must include the exact device screen resolution coordinates and an elapsed time variable (driven by a useSharedValue combined with a continuous withRepeat animation loop).[1]
3. Within the SKSL C++-like syntax, calculate advanced noise algorithms—such as Simplex noise or Perlin noise—based on the normalized canvas coordinates.[1]
4. Map the output of these noise functions directly to the Hourglass Deep Space Palette, forcing the pixels to continuously interpolate between Deep Eggplant (#0D0C14), Cyan (#00C2FF), and Violet (#A78BFA), creating the undulating, auroral effect beneath the

interface.[1]

Declarative Skia Gradient Alternative

If writing raw SKSL mathematics is deemed excessively complex for the project's long-term maintenance lifecycle, React Native Skia provides declarative primitives that can simulate the mesh effect perfectly.[1]

1. Wrap the application's root background in a Skia <Canvas> spanning flex: 1.
2. Instantiate three to four distinct <RadialGradient> or <SweepGradient> components within the canvas.[1]
3. Bind the c (center coordinate vectors) and r (radius) parameters to Reanimated shared values.[1]
4. By animating these values utilizing complex, out-of-phase sine wave functions, the localized color nodes will physically orbit and drift across the background, successfully recreating the flowy mesh without manual shader programming.[1]

Phase III: Architecting the Liquid Glass Surfaces

With the underlying spatial mesh active, the flat, opaque containers currently comprising the analytical cards must be eradicated and replaced with translucent "Liquid Glass" surfaces that refract the moving mesh below.[1]

The migration strategy here demands strict platform awareness to optimize for both visual fidelity and thermal device performance.

Platform-Specific Rendering Strategies

When a native operating system BlurView attempts to continuously sample and blur an underlying animated GPU-driven Skia background, the cross-thread communication often bottlenecks, leading to hard crashes on Android or severe frame drops.[1]

The following table dictates the routing of the liquid glass rendering pipeline based on the target OS:

Target Platform and Version	Recommended Rendering Component	Technical Justification and Performance Profile
iOS 26 and Above	expo-glass-effect or @callstack/liquid-glass	Utilizes the brand new native Apple UIGlassEffect API.[17] Provides genuine optical refraction, real-time magnification, and dynamic

		specular highlights that shift with the hardware gyroscope. The absolute pinnacle of the tech aesthetic.[4]
Legacy iOS (Below 26)	Skia <BackdropFilter>	Provides stable 60+ FPS performance. Avoids the native bridge by executing the ImageFilter.Blur(16, 16) directly within the same Skia canvas handling the background mesh.[1]
All Android Devices	Skia <BackdropFilter>	Android inherently lacks robust native implementations for real-time background blurring of dynamic elements.[1] Utilizing Skia ensures identical aesthetic parity with legacy iOS without crashing the system.[1]

Strict Performance Governance

Deploying computationally intensive translucent materials necessitates uncompromising performance governance to maintain a 60-120 FPS rendering budget.[1]

1. **Layer Limitation**: The primary architectural constraint is the strict limitation of glass layers. A maximum of three overlapping liquid glass layers may be permitted on any single viewport.[1] Deeply nesting a blurred badge inside a blurred card inside a blurred modal will compound the GPU workload exponentially and trigger thermal throttling.[1]
2. **Android Texture Caching**: For Android deployments utilizing the Skia fallback, the application of the renderToHardwareTextureAndroid={true} property is absolutely mandatory. This directive forces the Android rendering engine to offload the component to the GPU, caching it as a static hardware texture and preventing continuous, disastrous CPU-level repainting during scroll events.[1]
3. **Opacity Thresholds**: Developers must strictly ensure that the opacity values applied to the glass view itself never drop below 1.0. Utilizing sub-1.0 opacities on a blurring component is a known catalyst for rendering pipeline glitches and visual artifacting.[1]

Surface Detailing: Borders and Simulated Thickness

To satisfy the Version 2.0 requirement for edge illumination, developers must engineer hardware-accelerated gradient borders framing the Skia BackdropFilter.[1]

1. Utilize @react-native-masked-view/masked-view to instantiate a base LinearGradient spanning the exact dimensions of the target card.[1]
2. Construct a structural mask utilizing two intricately nested views. The outer view defines the absolute perimeter and border radius (e.g., 16px). Nested immediately inside is an inner view, offset by a microscopic margin representing the border width (strictly calculated at 1 to 1.5 pixels).[1]
3. This masking exclusion forces the rendering engine to display the underlying bright neon gradient only through the microscopic gap, generating a mathematically perfect, luminous perimeter.[1]

To simulate physical thickness, developers must integrate react-native-inner-shadow. This utilizes Skia's advanced canvas painting capabilities to draw highly precise inset shadows—painting a high-opacity black (rgba(0,0,0,0.6)) at the top internal edge, combined with a low-opacity white highlight (rgba(255,255,255,0.1)) along the bottom edge, achieving a tactile, beveled appearance without cluttering the Document Object Model (DOM) with absolute-positioned transparent overlays.[1]

Phase IV: High-Performance Data Visualization Integration (Victory Native XL)

Technical applications inherently demand the display of complex analytics.[1] The current data visualizations within the Hourglass application are rendered as flat, solid, vector graphics lacking depth and fluidity.[1] Embedding flat charts into a sophisticated, glassmorphic environment shatters the aesthetic immersion.[1]

The migration specification mandates the complete replacement of existing charting architecture with Victory Native XL (v41 or higher).[1] Victory Native XL entirely shifts the mental model of chart construction. Instead of forcing the React Native bridge to render hundreds of individual SVG nodes, it mathematically calculates data coordinates directly in JavaScript and renders a single, highly optimized Skia Path directly on the GPU canvas.[1]

The following technical implementations must be executed across the application's core screens:

1. The "THIS WEEK" Cylindrical Bar Chart

The legacy chart utilizes solid, flat green rectangles.[1]

- **Implementation**: This must be rewritten utilizing the <CartesianChart> wrapping a custom

<Bar> component.[26]

- **Aesthetic Detail**: Instead of a solid fill, the bars must be injected with a vertical Skia LinearGradient, fading from neon green at the peak to a semi-transparent green at the baseline.[1] Applying an inner shadow to the bar's RRect (Rounded Rectangle) provides a cylindrical, volumetric appearance aligning with the physical glass UI.[1]

2. The Line Charts ("AI TRAJECTORY" & "WEEKLY EARNINGS")

The legacy charts rely on simple, unstyled flat line strokes.[1]

- **Implementation**: In Victory Native XL, the <Line> component takes a PointsArray prop (points.y) and returns a Skia path.[1]
- **Aesthetic Detail**: To achieve the neon tech aesthetic, these paths must emit simulated light. By passing a DropShadowImageFilter or BlurMaskFilter into the Skia Paint properties applied to the stroke, the data line gains an ethereal, luminous glow.[1]
- **Area Fills**: The space beneath the lines cannot remain empty. Developers must utilize the <Area> component. By defining a <LinearGradient> within a <Defs> block and applying it via the color or children prop of the <Area> component, the chart creates a gradient transitioning from the highly saturated brand color at the top edge of the data line down to complete transparency at the y0 (bottom) axis.[1]

3. The "PRIME RADIANT" Projection Chart

For tracking projections versus actuals, the standard line chart is insufficient.

- **Implementation**: Developers must utilize the <AreaRange> component.[1]
- **Aesthetic Detail**: By passing both an upper and lower PointsArray bound (e.g., y and y0), the chart generates a shaded, translucent boundary visualizing error bounds and confidence intervals with profound aesthetic sophistication, perfectly aligned with AI predictive modeling.[1]

4. The Circular Progress Ring ("90% AI USAGE")

The legacy ring is an entirely static, solid bi-color stroke.[1]

- **Implementation**: This must be completely rebuilt using a standard Skia <Path> stroked with a <SweepGradient>.[1]
- **Aesthetic Detail**: The gradient must transition angularly through the application's secondary palette (Cyan to Violet to Magenta) anchored to the center of the canvas.[1] Crucially, the strokeEnd property of the path must be driven by a Reanimated useSharedValue. When the user navigates to the screen, this value must animate from 0 to 0.90 utilizing a physics-based spring configuration to demonstrate momentum and fluidity.[1]

Phase V: Motion Choreography and Spatial Navigation

Static aesthetics, regardless of their high visual fidelity, instantly shatter the illusion of a modern tech environment upon user interaction.[1] Fluidity must be deeply engineered into every state change, utilizing React Native Reanimated 4.2.1 to bypass the inherent bottlenecks of the legacy asynchronous Animated API.[1]

Physics-Based Micro-Interactions

In a modern spatial UI, linear timing transitions (moving at a constant mathematical speed) feel distinctly mechanical and outdated.[1] Digital elements must behave as if they possess physical mass, momentum, and friction.[1]

- When a liquid glass card mounts onto the screen (e.g., navigating to the Overview analytical stack), the cards must not merely fade into existence. They must utilize a staggered spatial reveal, animating dimensional scale, opacity, and Y-axis translation simultaneously using spring functions.[1]
- List items must apply a timing delay, cascading downward exactly 150 to 300 milliseconds after the previous item, creating a satisfying "settling in" effect.[1]
- Interactive buttons must provide instantaneous tactile feedback, dynamically increasing their inner shadow opacity and physically compressing in scale (e.g., scale(0.96)) upon a PressIn gesture.[1]

Shared Element Transitions (SET)

The navigation paradigm in a flowy application must aggressively eliminate harsh, traditional screen cuts.[1] When a user taps the "EARNINGS" glass card on the Home dashboard to view deeper details, the card should physically detach from the layout and seamlessly expand across the z-axis into the header of the destination screen.[1]

Reanimated 4.2.1 introduces native, deeply integrated support for Shared Element Transitions (SET) specifically optimized for the New Architecture.[6]

- **Enablement**: Because SET is currently an experimental feature, the engineering team must strictly define "ENABLE_SHARED_ELEMENT_TRANSITIONS": true as a static feature flag within the project's package.json.[6]
- **Implementation Syntax**: Developers simply assign an identical string to the sharedTransitionTag property on a specific <Animated.View> on the origin screen and the corresponding <Animated.View> on the destination screen.[33]
- **Execution**: The Reanimated engine detects the stack change, creates a temporary third view displayed over the hierarchy, and calculates the complex geometric interpolation required to morph the component's dimensions and styling.[33] Because this calculation occurs on the native C++ layer, the transition remains perfectly fluid at 60 FPS while the JavaScript thread mounts the complex DOM of the new screen in the background.[1]

Anchoring with Native Tabs via Expo Router

Finally, to completely solidify the premium, native feel of the application, the architecture must abandon standard JavaScript-based bottom tab navigators.[34]

Expo SDK 55 and Expo Router v7 introduce the revolutionary <NativeTabs> API.[2] By utilizing the expo-router/unstable-native-tabs import, developers can construct a file-based routing layout that compiles down to true, platform-native tab bars on both iOS and Android.[34] Crucially, on iOS 26, this API deeply integrates with the UIGlassEffect native engine out-of-the-box, ensuring that the primary navigation chrome behaves with the exact same premium refractive optical properties as the application's internal data cards, creating a unified, unbroken spatial experience.[36]

4. Conclusion

The modernization of the Hourglass application is a rigorous, highly technical exercise in marrying strict mathematical data visualization with advanced graphics engineering. By executing this comprehensive migration specification, the application transcends the superficial constraints of traditional flat CSS styling and DOM bridging.

The transition to Version 2.0 of the Brand Guidelines replaces opaque, static branding with a dynamic, light-reactive spatial identity governed by physical rules. The strategic deployment of React Native Skia for SKSL fragment shaders and hardware-accelerated BackdropFilters creates an immersive, living environment cross-compatible on both Android and iOS.[1] Integrating Victory Native XL ensures that mission-critical analytical data is rendered with unprecedented volumetric depth and GPU efficiency.[5] Finally, choreographing the user experience with Reanimated 4.2.1 Worklets, Shared Element Transitions, and Expo SDK 55's Native Tabs guarantees that the application moves with the physical fluidity expected of top-tier products in 2026.[1]

The resulting architecture not only achieves the requested "flowy, dark glass, tech" aesthetic, but establishes a highly performant, future-proof foundation capable of scaling seamlessly alongside rapidly evolving mobile hardware capabilities.

Works cited

1. Revamping App Design with Brand Guidelines.pdf
2. What's new in Expo SDK 55 - YouTube, accessed March 19, 2026, https://www.youtube.com/watch?v=q72aeXsbF9c
3. React Native 0.83 - React 19.2, New DevTools features, no breaking changes, accessed March 19, 2026, https://reactnative.dev/blog/2025/12/10/react-native-0.83
4. Implementing Liquid Glass UI in React Native: Complete Guide 2025 | Cygnis, accessed March 19, 2026, https://cygnis.co/blog/implementing-liquid-glass-ui-react-native/
5. Introduction | Victory Native - Nearform, accessed March 19, 2026,

https://nearform.com/open-source/victory-native/docs/
6. Introducing React Native Reanimated 4.2.0 | Shared Element Transitions Are Here! | Software Mansion, accessed March 19, 2026, https://blog.swmansion.com/introducing-reanimated-4-2-0-71eea21ca861
7. Best google font pairings for UI design in 2025 | by Matt Medley | Bootcamp - Medium, accessed March 19, 2026, https://medium.com/design-bootcamp/best-google-font-pairings-for-ui-design-in-2025-ba8d006aa03d
8. 8 top font pairing examples for eye-catching web design - Webflow, accessed March 19, 2026, https://webflow.com/blog/font-pairing
9. What's New in Expo SDK 55 - Medium, accessed March 19, 2026, https://medium.com/@onix_react/whats-new-in-expo-sdk-55-6eac1553cee8
10. Making AI feel human in a mobile app with Expo, Reanimated, and Skia, accessed March 19, 2026, https://expo.dev/blog/making-ai-feel-human-in-a-mobile-app-with-expo-reanimated-and-skia
11. Mesh Gradient In React Native with Expo : r/reactnative - Reddit, accessed March 19, 2026, https://www.reddit.com/r/reactnative/comments/1fddlhy/mesh_gradient_in_react_native_with_expo/
12. Android — MeshGradient — Coming Soon | by Akshay Nandwana | Feb, 2026 | Medium, accessed March 19, 2026, https://medium.com/@anandwana/android-meshgradient-coming-soon-d0c45fd8c783
13. Animated Gradient Background with React Native Skia | by Serhii Husachenko - Medium, accessed March 19, 2026, https://medium.com/@shusachenko/animated-gradient-background-with-react-native-skia-8c9b091c5337
14. Gradients | React Native Skia - Shopify, accessed March 19, 2026, https://shopify.github.io/react-native-skia/docs/shaders/gradients/
15. Animated Gradient in React Native (Skia) - Reactiive, accessed March 19, 2026, https://reactiive.io/articles/animated-gradient
16. React Native Image Optimization: Performance Essentials | by Engin Bolat - Medium, accessed March 19, 2026, https://medium.com/@engin.bolat/react-native-image-optimization-performance-essentials-9e8ce6a1193e
17. How To Use Liquid Glass in React Native - Callstack, accessed March 19, 2026, https://www.callstack.com/blog/how-to-use-liquid-glass-in-react-native
18. Expo GlassEffect - Expo Documentation, accessed March 19, 2026, https://docs.expo.dev/versions/latest/sdk/glass-effect/
19. Backdrop Filters | React Native Skia - Shopify, accessed March 19, 2026, https://shopify.github.io/react-native-skia/docs/backdrops-filters/
20. @sbaiahmed1/react-native-blur - npm, accessed March 19, 2026, https://www.npmjs.com/package/@sbaiahmed1/react-native-blur
21. GitHub - iway1/react-native-gradient-border, accessed March 19, 2026,

https://github.com/iway1/react-native-gradient-border
22. Actual Gradient Borders in React Native (the almighty Masked View) - DEV Community, accessed March 19, 2026, https://dev.to/iway1/actual-gradient-borders-in-react-native-the-almighty-masked-view-272g
23. ShinMini/react-native-inner-shadow - GitHub, accessed March 19, 2026, https://github.com/ShinMini/react-native-inner-shadow
24. Box | React Native Skia, accessed March 19, 2026, https://shopify.github.io/react-native-skia/docs/shapes/box/
25. Area (Component) | Victory Native - Nearform, accessed March 19, 2026, https://commerce.nearform.com/open-source/victory-native/docs/cartesian/area/
26. Basic Bar Chart | Victory Native - Nearform, accessed March 19, 2026, https://commerce.nearform.com/open-source/victory-native/docs/cartesian/guides/basic-bar-chart/
27. Line (Component) | Victory Native - Nearform, accessed March 19, 2026, https://commerce.nearform.com/open-source/victory-native/docs/cartesian/line/
28. Shadows | React Native Skia - Shopify, accessed March 19, 2026, https://shopify.github.io/react-native-skia/docs/image-filters/shadows/
29. Creating Victory charts in React Native - LogRocket Blog, accessed March 19, 2026, https://blog.logrocket.com/creating-victory-charts-react-native/
30. AreaRange (Component) | Victory Native - Nearform, accessed March 19, 2026, https://nearform.com/open-source/victory-native/docs/cartesian/area/area-range/
31. Animated - React Native, accessed March 19, 2026, https://reactnative.dev/docs/animated
32. Dec 5 2025: Issue 8 - Nativeweekly - Beehiiv, accessed March 19, 2026, https://nativeweekly.beehiiv.com/p/dec-5-2025-issue-8
33. Shared Element Transitions | React Native Reanimated, accessed March 19, 2026, https://docs.swmansion.com/react-native-reanimated/docs/shared-element-transitions/overview/
34. Expo Router v55: more native navigation, more powerful web, accessed March 19, 2026, https://expo.dev/blog/expo-router-v55-more-native-navigation-more-powerful-web
35. Bottom tab navigation in Expo Router with authentication | by Fredrik Burmester | Medium, accessed March 19, 2026, https://medium.com/@fredrik.burmester/bottom-tabs-in-expo-router-with-authentication-acf7f7edee6d
36. Native tabs - Expo Documentation, accessed March 19, 2026, https://docs.expo.dev/router/advanced/native-tabs/
37. Expo Router iOS 26 Tutorial: Native Tabs, Toolbars, Zoom Transitions (SDK 55) - YouTube, accessed March 19, 2026, https://www.youtube.com/watch?v=_XGIN4jvIaA